

MLB Home Run Predictor

Grayson, Joey, Jeffrey, and Aidan



Overview

What we're doing

- Using *real MLB Statcast data*
- Python-based **machine learning pipeline**

Why it matters

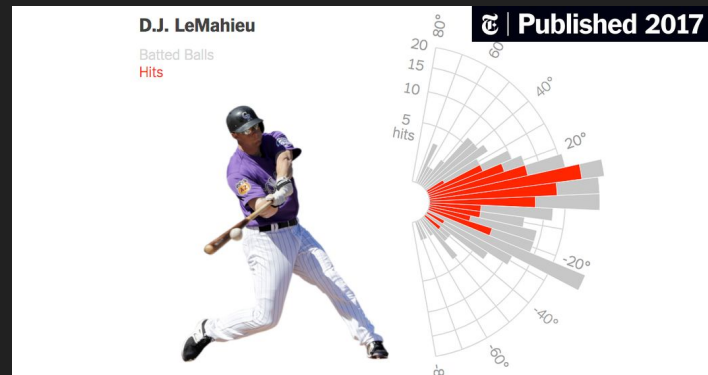
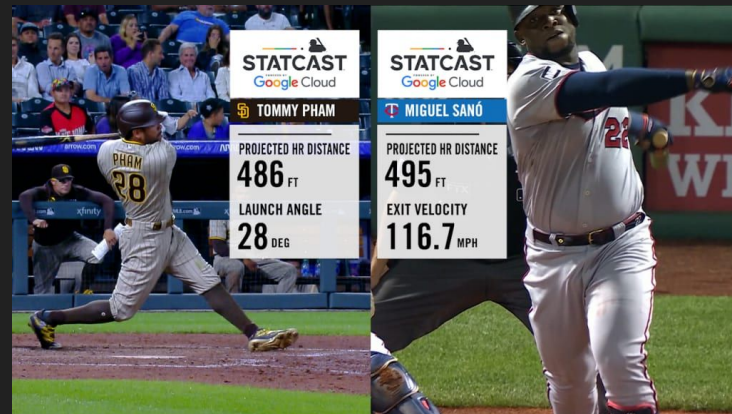
- We wanted to see if we could predict one of the most exciting plays in baseball with simple machine learning

What goes into the model

- Hitter metrics (who's at the plate)
- Pitch characteristics (speed, type, location)
- Ball-tracking variables (exit velo, launch angle, etc.)

Our goal

- Build a **simple but effective** home run predictor
- Use data science to understand something as **explosive and “random”** as a home run



Understanding the problem

Item 1

A home run is influenced by many interacting factors: exit velocity, launch angle, pitch type, ballpark effects, and even the count. This complexity makes HR prediction challenging.

Item 2

We needed to identify which variables matter most. Using Statcast gave us high-resolution tracking data so we could isolate features like:

- **Exit Velo (EV)**
- **Launch Angle (LA)**
- **Pitch Location & Speed**

Item 3

The key challenge was avoiding noise: tons of batted balls look similar but have different outcomes. Our model needed to generalize rather than memorize.

Project objective:

Build a lightweight machine learning classifier in Python that predicts the probability of a home run given the metrics of a single batted ball.

Our goals:

- Ingest and clean Statcast data efficiently
- Engineer meaningful features (EV, LA, spray angle, etc.)
- Train and evaluate a model using scikit-learn
- Visualize results and highlight which factors drive HR outcomes

The Data

Dataset Source

- Kaggle competition: *Predict Home Runs with MLB Pitch-By-Pitch Data*
- Thousands of pitches from real MLB games
- Clean CSV files = no scraping or API setup

What Each Row Is

- One **pitch** thrown in an MLB game
- Pitch info: speed, type, spin, horizontal & vertical location
- IDs: batter ID, pitcher ID
- Game context: count, inning, balls/strikes

Target Label

- `is_home_run = 1` = pitch contact resulted in a home run
- `is_home_run = 0` = anything else

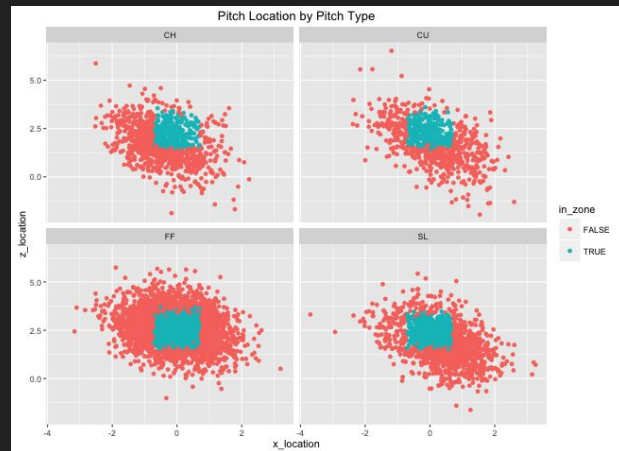
Why This Dataset

- Easy to load straight into Python
- Great fit for a simple ML classifier
- Real Statcast-style metrics with almost no preprocessing

Prepping the Data

Initial Data Preparation

- Dropped rows with missing values
- Kept only pitches that resulted in **contact**
- Encoded categorical features (e.g., **pitch type**)
- Standardized numeric features (speed, spin, location, etc.)



Why Pitch-Level Data Works

- Bad pitches = **higher probability** of a home run
- Middle-middle locations + high exit speed often share similar pitch patterns
- Even **simple pitch features** provide strong predictive power for home runs

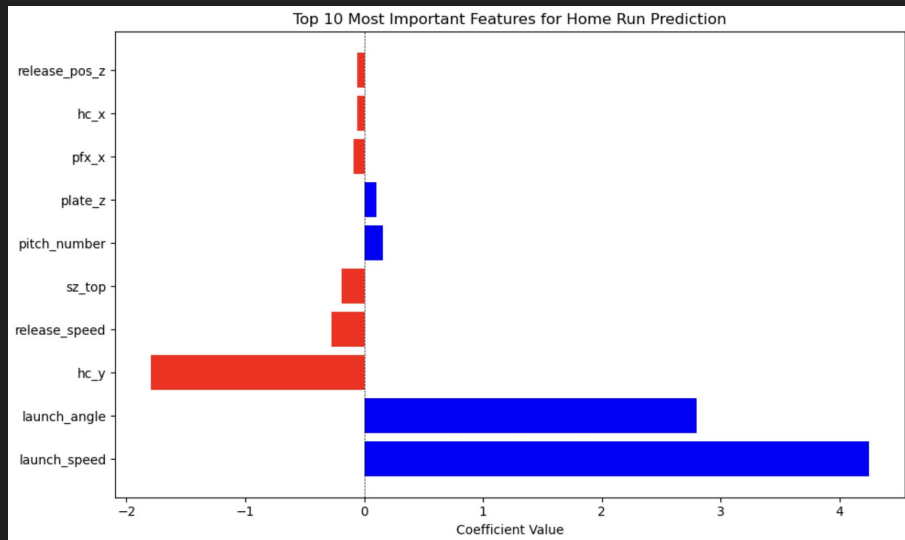
Features

Features Used in the Model

- **Pitch speed** – Faster pitches can turn solid contact into higher exit velocity.
- **Pitch type** – Categorical feature (fastball, slider, changeup, etc.) to capture pitch style.
- **Horizontal location** (plate_x) – Inside vs. outside changes how much power a hitter can generate.
- **Vertical location** (plate_z) – Middle, up, or down in the zone behave differently for home runs.
- **Count** (balls/strikes) – Hitters swing more aggressively in hitter's counts.
- **Batter ID and pitcher ID** – Encoded to capture individual player tendencies.

Why these features matter

- They explain most of the variation behind hard-hit balls
- Light enough for simple models like Logistic Regression
- Still capture meaningful baseball strategy patterns



The Model

Model Pipeline

- **Train/Test Split**
 - Split the dataset to test how well the model generalizes to **new pitches**.
- **One-Hot Encoding**
 - Converted categorical variables (like **pitch type**) into numeric columns the model can use.
- **Feature Scaling**
 - Standardized numeric features (speed, spin, location) to keep everything on a **comparable scale**.
- **Classifier: Logistic Regression**
 - Simple, interpretable **binary classifier** that predicts: home run (1) vs. not a home run (0).

Why Logistic Regression?

- Very **fast to train** and works well on structured tabular data
- **Coefficients are interpretable**, shows which pitch characteristics raise HR risk
- Less prone to **overfitting** than more complex models
- Great **starter model** for binary outcomes like “HR vs. not HR”

What We Wanted to Learn

- Which **pitch features** influence home run probability the most
- How well a **simple model** can separate “dangerous” pitches from safer ones
- Whether **pitch characteristics alone** provide reliable signals for predicting home runs

The Next Steps

Data Expansion

- Pull every pitch Aaron Judge has seen since his MLB debut
- Pull every pitch Shohei Ohtani has seen since his MLB debut
- Filter by pitcher handedness (RHP vs LHP)
- Organize pitches into velocity buckets (slow / average / high)
- Compute each player's historical HR rate in each matchup scenario
- Build a clean dataset that links: handedness + pitch speed into HR outcomes

Model Weighting & Prediction

- Use real career HR rates as baseline probabilities
- Weight predictions using Judge/Ohtani's actual HR tendencies
- Compare matchup-specific risk vs overall baseline
- Generate a game-level HR likelihood based only on: pitcher handedness + pitch velocity
- Move toward a simple user tool that outputs the chance of homer today assuming 4 PA



Results & Takeaways

- We were able to estimate the chance that Judge or Ohtani would hit a home run, and even adjust it using the betting line.
- Pitch speed and pitcher handedness clearly affect HR odds, especially for big power hitters.
- But predicting home runs is extremely hard since so many things impact them: pitch location, swing path, ballpark, weather, and pure randomness.
- Building a truly accurate model would require huge amounts of detailed Statcast data and a much more advanced approach.
- Overall, our project shows we can point in the right direction, but home runs are such rare, unpredictable events that perfectly modeling them is still a major challenge.

Aaron Judge vs RHP

Raw per-PA HR Prob: 17.59%

League HR/PA vs R: 3.50%

Calibrated per-PA HR Prob: 8.44%

Expected HRs over 4 PAs: 0.34

Game HR Distribution:

0 HR: 70.28%

1 HR: 25.91%

2 HR: 3.58%

3 HR: 0.22%

4 HR: 0.01%

Aaron Judge vs LHP

Raw per-PA HR Prob: 16.15%

League HR/PA vs L: 4.00%

Calibrated per-PA HR Prob: 7.98%

Expected HRs over 4 PAs: 0.32

Game HR Distribution:

0 HR: 71.69%

1 HR: 24.88%

2 HR: 3.24%

3 HR: 0.19%

4 HR: 0.00%

Limitations and Future

Limitations

- **Severe class imbalance:** Home runs are rare, so simple models struggle to learn the positive class.
- **Pitch-only inputs:** The Kaggle file is pitch-by-pitch, so we didn't use exit velocity, launch angle, or spray angle.
- **No contextual factors:** We didn't include ballpark dimensions, weather, home vs. away, or day vs. night, even though they clearly affect HR odds.
- **Model ceiling:** Logistic Regression is easy to interpret, but not the most powerful for nonlinear baseball relationships.

Future Improvements

Richer context features

- Add ballpark info, home vs. away, and day vs. night to capture how environment affects HR probability.

More players & pitcher-specific inputs

- Expand the model so users can choose any pitcher and more hitters, this would require much more Statcast data to stay accurate.

Stronger models

- Try XGBoost or Random Forest to learn more complex interactions between features.

Visual insights

- Build HR heatmaps and pitch risk zone plots to show where dangerous pitches live in the strike zone.

Questions